

ENKKU ICPC Contest 2025

Editorial

เอกสารนี้เรียบเรียงเฉลยของโจทย์ในการแข่งขันตามความยากง่าย(ในความเห็นของผู้
ออกโจทย์) จากข้อที่ง่ายไปยังข้อยาก

ถ้าหากไม่อยากถูกสปอย สามารถใช้ลิงค์ด้านล่างนี้ในการกระโดดไปยังข้อที่สนใจได้

[Problem A: Adaptive](#)

[Problem B: Barsino](#)

[Problem C: Concrete](#)

[Problem D: Desire](#)

[Problem E: Express](#)

[Problem F: Foresee](#)

[Problem G: Goodness](#)

[Problem H: H.Uncle](#)

Problem H: H.Uncle

ข้อนี้ค่อนข้างจะตรงไปตรงมา สามารถแก้ได้ด้วยการ for loop 3 ชั้น หาคัดที่จะแยกแต่ละ string เพื่อทำการหา encoded message ที่เป็นไปได้ทุกรูปแบบ และเลือก string ที่มีมาก่อนในลำดับพจนานุกรม

for i, j, k as the indices to break the string

encoded_string = reverse(s[0:i]) + reverse(s[i:j]) + reverse(s[j:k]) + reverse(s[k:n])

Running time: $O(n^4)$ ซึ่งสามารถรันได้ทันสำหรับ $n = 75$

Problem A: Adaptive

สังเกตว่าเราสามารถที่จะสร้าง grid ที่ประกอบไปด้วยเลข 1 ทั้งหมด ยกเว้นตำแหน่งบน ขวาสุด กับตำแหน่งบนซ้ายสุด เพื่อที่จะไม่ต้องกังวลกับการหา shortest path บน grid นี้

??	1	1	1	1	1	??
1	1	1	1	1	1	1
1	1	1	1	1	1	1
1	1	1	1	1	1	1

ซึ่งสามารถทำได้เสมอ ถ้า T_A และ T_B มีค่าอย่างน้อย $R + C - 1$

Special Case: ถ้า $R = 1$ หรือ $C = 1$

จะทำให้ grid กลายเป็นแถว(หรือคอลัมน์)เดียว

??	1	1	1	1	1	??
----	---	---	---	---	---	----

ในกรณีนี้นอกจากเงื่อนไขที่ต้องเช็คด้านบนแล้ว เราต้องเช็คเพิ่มด้วยว่า $T_A = T_B$ หรือไม่ เนื่องจาก path จากซ้ายบนไปขวาล่าง และ path จากขวาบนไปซ้ายล่าง จะกลายเป็น path เดียวกัน

Running time: $O(1)$

Problem E: Express

ข้อนี้ สามารถใช้วิธีการแก้แบบตรงไปตรงมาได้ คือ

1. การหา shortest path บนกราฟตั้งต้น
2. for loop สำหรับในแต่ละ edge ทำการเพิ่มระยะทางบน edge นั้นๆ และหา shortest path ใหม่อีกครั้ง

รวมแล้วจะเป็นการรัน shortest path algorithm ทั้งหมด $(M + 1)$ ครั้ง ซึ่งสามารถรันได้ทันเวลาเนื่องจาก constraints $1 \leq N, M \leq 1,000$

Running time: $O(MN \log(N))$

วิธีที่เร็วขึ้นไปอีก

อีกวิธีหนึ่งที่สามารถทำได้ คือการรัน shortest path บนกราฟตั้งต้น และทำการเก็บ second best path สำหรับทุกๆ vertex บนกราฟ แล้วทำการ backtrack ดูว่า edge ไหนบ้างบน shortest path ที่สามารถที่จะทำการเลี้ยวแล้วไปทางเส้นทางอื่นได้ (second best option) โดยที่ระยะทางรวมบวกเพิ่มไม่เกิน 1 หน่วย

ดูศึกษาเพิ่มเติมได้ที่ C_fast.cpp

Running time: $O(N \log(N))$

Problem F: Foresee

ข้อสังเกตสองข้อ

1. ถ้าหากไม่มี special round และไฟที่มีค่าสูงกว่าชนะเสมอ เราสามารถใช้ Greedy algorithm ได้
 - ในแต่ละรอบ เราสามารถเลือกไฟที่ต่ำที่สุดที่ยังมีค่ามากกว่าไฟที่คู่ต่อสู้ใช้
 - แต่ถ้าไม่มีทางชนะอยู่แล้ว เราอาจจะลงไฟที่ค่าต่ำสุดแทน
2. เนื่องจากมี special round ที่ไฟที่มีค่าต่ำกว่าชนะ สังเกตได้ว่า เมื่อเริ่มเกมมา เราสามารถจะเอาไฟสองใบที่มีค่าต่ำที่สุดในมือเรา มาเตรียมไว้ได้เลยว่าจะใช้สำหรับ special round นี้เท่านั้น เนื่องจากไฟสองใบนี้ไม่มีค่าใดๆสำหรับรอบธรรมดา

ดังนั้น สรุป algorithm ที่เราจะใช้ได้ดังนี้

1. เริ่มต้นมา เก็บไฟสองใบต่ำสุดเพื่อใช้ใน special round
2. ใช้ไฟที่เหลือในการเล่นรอบปกติ ด้วยวิธี Greedy (เลือกไฟที่ต่ำที่สุดที่ยังมีค่ามากกว่าไฟที่คู่ต่อสู้ใช้)
3. สำหรับรอบ special round ก็แก้ได้ด้วยวิธี Greedy เช่นกัน (เลือกไฟที่สูงที่สุดที่ยังมีค่าน้อยกว่าไฟที่คู่ต่อสู้ใช้)

Running time: $O(N)$

Problem G: Goodness

ข้อนี้สามารถแก้ได้ด้วยวิธี Greedy algorithm

1. ถ้า node ไหนที่ไม่มี edge โยงเข้า (in-degree = 0) มาหาเลย เราสามารถที่จะให้ node นั้น เป็น GP ได้เลย และ node ที่โดน node นั้นโยงไปหา ก็จะเป็น NSG
2. หลังจากที่ทำตามข้อ 1 ไปเรื่อยๆ อาจจะมีจุดหนึ่งที่ไม่ได้มี node ไหนที่มี in-degree เป็นศูนย์แล้ว เนื่องจากเกิด Cycle ขึ้น
 - ในกรณีนี้เราสามารถเลือก node ไหนก็ได้ให้เป็น NSG และลบออกจากกราฟ
 - กลับไปทำข้อ 1 ต่อ อีกครั้ง

ทำตามนี้ไปเรื่อยๆ จนกว่าทุก node จะถูก assign GP/NSG

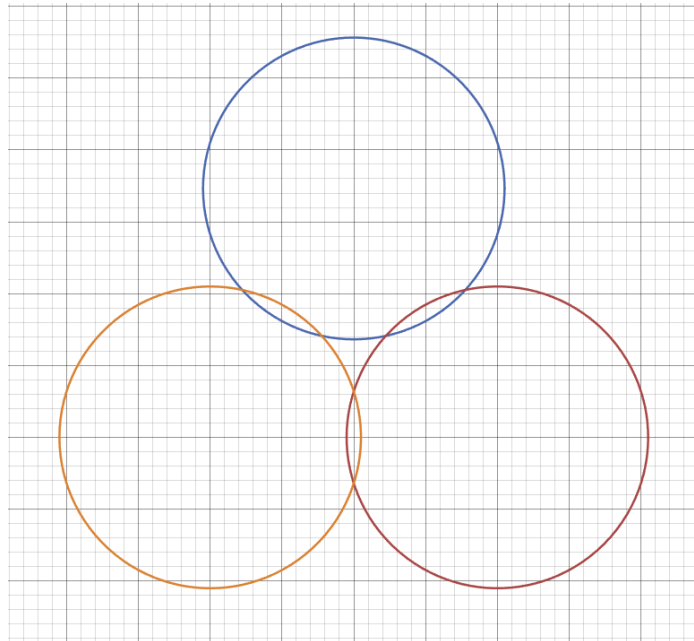
Running time: $O(N)$

Problem B: Barsino

ข้อนี้สามารถแปลความหมายได้อีกคือ ต้องการเช็คว่างกลม 3 วงนั้นมีการ intersect กัน พร้อมกันทั้งสามวงไหม

ผู้เข้าแข่งขันส่วนใหญ่เข้าใจผิดไปว่าข้อนี้สามารถเช็คได้ง่ายๆ โดยการดูว่าจุดศูนย์กลางของทุกคู่หนึ่งห่างกันไม่เกิน $2R$ หรือไม่

นี่คือ counterexample ของสมมติฐานนี้



จะเห็นได้ว่าทุกคู่จุดศูนย์กลางห่างกันไม่เกิน $2R$ แต่เราจะเห็นว่าไม่มีช่องว่างอยู่ตรงกลาง ดังนั้นไม่มีส่วนไหนเลยที่อยู่ในวงกลมทั้งสามในเวลาเดียวกัน

วิธีที่ถูกต้อง คือจะเช็คว่าจุดตัดของวงกลมสองวง อยู่ภายในวงกลมอีกหนึ่งวงไหม ดังนี้

1. คำนวณหาจุดตัดของ Circle #1 and Circle #2 แล้วดูว่ามีจุดตัดไหนไหมที่อยู่ใน Circle #3 ถ้ามีตอบ “Rejected”
2. คำนวณหาจุดตัดของ Circle #1 and Circle #3 แล้วดูว่ามีจุดตัดไหนไหมที่อยู่ใน Circle #2 ถ้ามีตอบ “Rejected”
3. คำนวณหาจุดตัดของ Circle #2 and Circle #3 แล้วดูว่ามีจุดตัดไหนไหมที่อยู่ใน Circle #1 ถ้ามีตอบ “Rejected”

ถ้าไม่เข้าเคส 1 - 3, ตอบ “Passed”

ดังนั้นข้อนี้จึงต้องใช้สมการคณิตศาสตร์เข้ามาช่วยในการหาจุดตัด

Running time: $O(1)$

Problem D: Desire

วิธี Brute force ของข้อนี้คือ for loop 2 ชั้น เพื่อเช็คค่า คนที่ i กับคนที่ j มีความชอบตรงกันไหม ซึ่งเป็นวิธีที่ผู้เข้าแข่งขันส่วนใหญ่ทำมา แต่ว่ามี Running time $O(n^2)$ ซึ่งไม่สามารถรันได้ทัน เนื่องจาก constraint $N \leq 100,000$

ลอง reduce ปัญหาลง และพิจารณากรณีที่แต่ละคนมีหนังสือเพียงแค่ 1 เล่ม

ยกตัวอย่างประกอบ เช่น

คนที่ 1 ชอบ Book #2, คนที่ 2 ชอบ Book #1, คนที่ 3 ชอบ Book #2, คนที่ 4 ชอบ Book #3, คนที่ 5 ชอบ Book #2, คนที่ 6 ชอบ Book #2, คนที่ 7 ชอบ Book #3, คนที่ 8 ชอบ Book #3

ในกรณีนี้ แทนที่เราจะนั่งไล่ว่า คนที่ 1 ชอบเหมือนคนที่ 2 ไหม, คนที่ 1 ชอบเหมือนคนที่ 3 ไหม, คนที่ 2 ชอบเหมือนคนที่ 4 ไหม, etc.

เราสามารถจะถามว่า มีคนที่ชอบ Book #1 กี่คน, มีคนที่ชอบ Book #2 กี่คน, มีคนที่ชอบ Book #3 กี่คน แล้วนำมาคำนวณว่ามีกี่คู่ที่ชอบ Book #1, กี่คู่ที่ชอบ Book #2, กี่คู่ที่ชอบ Book #3 ซึ่งจะได้คำตอบคือ 4 Choose 2 + 3 Choose 2 = 6 + 3 = 9 ซึ่งจะทำให้ลด for จาก 2 ชั้น กลายเป็นชั้นเดียว

แล้วลองขยายปัญหาขึ้น เป็นแต่ละคนมีหนังสือที่ชอบ 2 เล่ม

ยกตัวอย่างประกอบ เช่น

คนที่ 1 ชอบ Book #(2,3), คนที่ 2 ชอบ Book #(1,5), คนที่ 3 ชอบ Book #(2,6), คนที่ 4 ชอบ Book #(3,7), คนที่ 5 ชอบ Book #(2,8), คนที่ 6 ชอบ Book #(2,3), คนที่ 7 ชอบ Book #(3,9), คนที่ 8 ชอบ Book #(3,10)

ถ้าหากเราใช้วิธีเดิมคือนับว่า มีกี่คู่ที่ชอบ Book #1, กี่คู่ที่ชอบ Book #2, กี่คู่ที่ชอบ Book #3, etc. เราจะได้ว่า 4 Choose 2 (สำหรับ Book #2) + 5 Choose 2 (สำหรับ Book #3) = 16 คู่ แต่ถ้าลองนั่งนับจริงๆแล้ว จะได้ว่ามีแค่ 15 คู่ ซึ่งขึ้นจากการนับเกินเนื่องจากมี คนที่ 1 ที่ชอบ Book #(2,3) และคนที่ 6 ที่ชอบ Book #(2,3) ดังนั้นจึงเกิดการนับจำนวนคู่ที่ชอบ Book#2 และจำนวนคู่ที่ชอบ Book #3 ซ้ำ

ดังนั้นเราจึงควรจะต้องลบจำนวนคู่ที่ชอบหนังสือสองเล่มเหมือนกันออก เพื่อให้ได้คำตอบที่ถูกต้อง นั่นคือ

(จำนวนคู่ที่ชอบหนังสือ i เหมือนกัน) - (จำนวนคู่ที่ชอบหนังสือ j,k เหมือนกัน)

กลับไปตอบคำถามหลักของเรา ที่แต่ละคนมีหนังสือที่ชอบ 3 เล่ม

โดยหลักการของ Inclusion-exclusion principle เราสามารถสรุปได้ว่าคำตอบของเราคือ

(จำนวนคู่ที่ชอบหนังสือ i เหมือนกัน)

- (จำนวนคู่ที่ชอบหนังสือ j,k) เหมือนกัน

+ (จำนวนคู่ที่ชอบหนังสือ x,y,z) เหมือนกัน

Running time: $O(N)$

Problem C: Concrete

ข้อสังเกตที่ 1: ลำดับของคอนกรีตในกลุ่มเดียวกันไม่มีผลต่อคำตอบ

ยกตัวอย่างเช่น

$N = 6, K = 2$

9, 4, 7, 13, 5, 1

ถ้าเราสลับตัวเลข 9 กับ 13 ที่สุดท้ายจะถูกจัดให้อยู่ในกลุ่มเดียว จะไม่ส่งผลต่อคำตอบ:

13, 4, 7, 9, 5, 1

ดังนั้น เพื่อความง่าย เราสามารถแปลงตัวเลขทั้งหมดให้กลายเป็นเลขกลุ่มได้เลย

เช่น จาก 9, 4, 7, 13, 5, 1 สามารถถูกแปลงให้เป็น 1, 3, 2, 1, 2, 3 ได้ เนื่องจาก 9 และ 13 จะถูกจัดให้อยู่ในกลุ่ม 1, 5 และ 7 จะถูกจัดให้อยู่กลุ่ม 2, 1 และ 4 จะถูกจัดให้อยู่กลุ่ม 3

ทำให้เราสามารถแปลงปัญหาให้กลายเป็นการขยับ 1, 3, 2, 1, 2, 3 ให้กลายเป็น

1, 1, 2, 2, 3, 3 โดยใช้จำนวนการขยับให้น้อยที่สุด

ข้อสังเกตที่ 2: โจทย์นี้สามารถแก้ได้โดยใช้ Longest Increasing (Non-decreasing) Subsequence

ปัญหากลายเป็นการหาลำดับ Non-decreasing ที่ยาวที่สุดเพื่อใช้เป็นแกนหลักที่จะไม่ขยับ ซึ่งเป็นปัญหา classic dynamic programming

เช่น 1, 1, 2, 3 เป็น longest non-decreasing subsequence ของ 1, 3, 2, 1, 2, 3

ดังนั้นเราต้องย้าย 2 ตัว

Running time: จริงๆ LIS สามารถทำได้ใน $O(N \log N)$ แต่สำหรับข้อนี้ $O(N^2)$ ก็สามารถรันได้ทันเนื่องจาก $N \leq 5,000$